COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures & Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	09/13/26	**DATE SUBMITTED:**	09/13/26
NAME:	Gabriel Dwyane D. Arididdon	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO 7.1 **Hands-on Activity 8.1 Sorting Algorithms Pt2**

1. PROCEDURE OUTPUT

Code + Console Screenshot	`const int maxSize = 10;`
	`int main() {`
	`  // create an array with unsorted list`
	`  int unsortedArr[maxSize];`
	`  //use time (0) as seed`
	`  srand(time(0));`
	`  //populate the array`
	`  for(int i = 0; i < maxSize; i++){`
	`  unsortedArr[i] = rand()%10;`
	`  }`

Observations	Since it is a random generated array, every time we run the code, it displays random values that is limited to the range 0-9' due to rand() %10.

Table 8-1. Array of Values for Sort Algorithm Testing

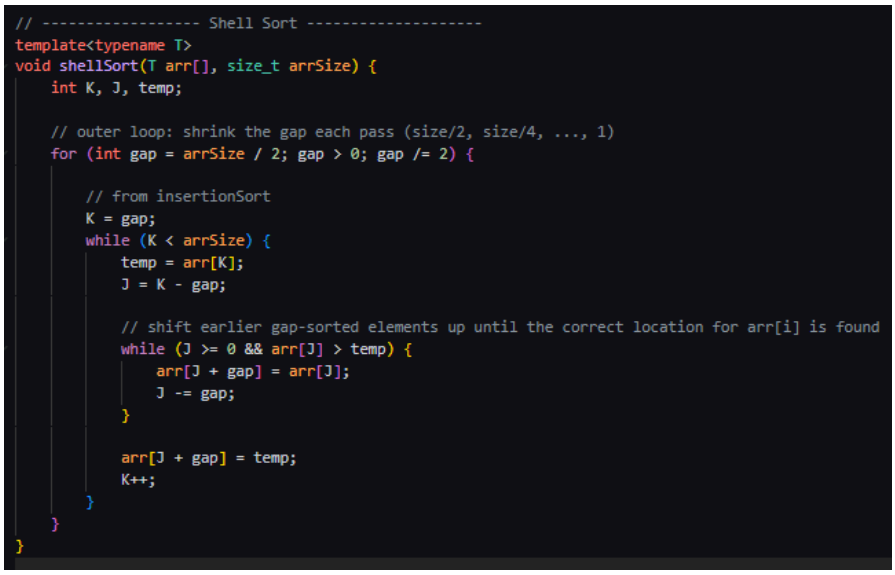
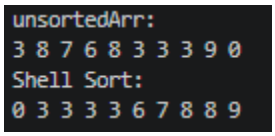
Code + Console Screenshot	 
Observations	In Shell Sort, it compares elements that are separated by a specific gap, usually starting with arrSize/2. The gap decreases each pass until it becomes 1, where Insertion Sort is used to compare and place the elements in their correct positions. The best-case time complexity is **O(n log n)** because the elements are already mostly sorted, so fewer shifts and comparisons are needed. The worst-case time complexity is **O(n²)** because the elements are in a very unfavorable order, causing many comparisons and shifts, especially when the gap becomes 1.

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```cpp
// ------------------ Merge Sort --------------------
template <typename T>
void merge(T arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;  //size of left sub-array
    int n2 = right - mid;     // size of right sub-array

    // Temporary arrays to hold sub-array data
    T* L = new T[n1];
    T* R = new T[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    // merge the temp arrays back into arr[left..right]
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // copy any remaining elements of L[]
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // copy any remaining elements of R[]
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}
```

```cpp
// Recursive helper that operates on index range [left, right]
template <typename T>
void mergeSortHelper(T arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSortHelper(arr, left, mid);       // sort first half
        mergeSortHelper(arr, mid + 1, right);  // sort second half
        merge(arr, left, mid, right);          // merge sorted halves
    }
}

// for calling the function
template <typename T>
void mergeSort(T arr[], size_t arrSize) {
    if (arrSize > 0)
        mergeSortHelper(arr, 0, arrSize - 1);
}
```

```
unsortedArr:
4 2 5 5 4 1 7 6 4 6
Merge Sort:
1 2 4 4 4 5 5 6 6 7
```

Observations	In Merge Sort, the array is recursively split into equal halves until single-element arrays remain, and then merged back together in sorted order. The bestand worst-case time complexity of Merge Sort is **O(N log N)** because the array is always divided in half regardless of whether it is already sorted or not. **mergeSort** - main caller **mergesortHelper** - It divides the array into two smaller parts. It finds the middle, then repeats the process for the left half and the right half until the array is split into smaller sections. **merge** - this function implements the conquer and combine concept. It combines two sorted parts by comparing their first elements. It puts the smaller element into the main array and continues until both parts are merged.

Table 8-3. Merge Sort Algorithm

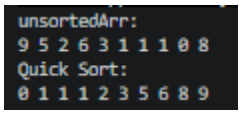
Code + Console Screenshot	 
Observations	In Quick Sort, the array is divided using a pivot. Smaller elements are placed on the left and larger elements on the right, then each part is sorted again. The best case is O(N log N) because the pivot divides the array into fairly balanced parts. The worst case is $O(N^2)$ because the pivot can repeatedly create very unbalanced parts, such as when one side has almost all the elements.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Yes, this works. The partition() step makes sure that once the pivot is placed, all elements on the left are ≤ the pivot, while all elements on the right are ≥ it. Because of this, the two sides can be sorted using any correct sorting algorithm, such as bubble, insertion, shell, or merge sort, and the final array will still be sorted correctly.

Array: {48, 21, 10, 15, 57, 29}, pivot = last element = 29
After one partition pass:
{15, 21, 10, 29, 57, 48}
↑ pivot in final position (index 3)
Left sub list: {15, 21, 10}
Right sub list: {57, 48}

Instead of calling quicksort() again on these two sub-lists, you could call insertionSort({15, 21, 10}) → {10, 15, 21} and insertionSort({57, 48}) → {48, 57}.
Combine:
{10, 15, 21, 29, 48, 57}

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

For this specific array, I would choose Shell Sort because there are only 15 elements. Since the array is small, Shell Sort can sort it quickly without needing too much extra memory or complicated steps. It also works directly on the array, so there is less overhead compared to algorithms that need extra processes like Merge Sort. For a small set of numbers like this, I think Shell Sort is the most practical choice.

Merge Sort and Quick Sort are both O(N log N) in their average or typical cases because they keep dividing the array into smaller parts. In Merge Sort, the array is repeatedly divided into left and right halves, then the parts are merged back together in order. In Quick Sort, the array is divided based on a pivot, with smaller values going to one side and larger values going to the other. Since the array keeps getting divided into smaller sections, there are about log N levels of division. Each level still needs around N operations to process the elements, so the total becomes O(N log N).

Conclusion

This activity taught me three advanced sorting algorithms: Shell Sort, Merge Sort, and Quick Sort. I learned that Shell Sort is basically an improved version of insertion sort using gaps, while Merge Sort and Quick Sort use the divide-and-conquer approach.

Implementing the three algorithms as function templates made them reusable and easier to use in main.cpp. Shell Sort was the easiest since I could modify my insertion sort code by adding gaps. Merge Sort and Quick Sort were more challenging because they use recursion and different index conditions. I also used AI assistance to help me understand some of the logic that I had difficulty with, especially the recursive process and partitioning in Merge Sort and Quick Sort. I still went through the code and made sure I understood how each part worked.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **My main challenge was the recursive parts of Merge Sort and Quick Sort, so I used AI assistance when I could not fully understand the logic.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.